

unordered_map 정리

정렬하지 않고 key-value를 해시 기반으로 빠르게 저장하는 컨테이너입니다.

사용처 (Where)

- 정렬이 필요 없는 빈도수 세기
- 빠른 key 검색
- 문자열/번호 기반 조회

방법 (How to Use)

- unordered_map<Key, Value>로 선언
- m[key]로 접근
- find로 존재 확인
- 순회 순서는 보장되지 않음

기본 정보

- 필요 헤더: #include <unordered_map>
- 기본 선언: unordered_map<string, int> cnt;

왜 써야 할까? (Why)

- map보다 평균 검색/삽입이 빠름
- 정렬이 필요 없는 빈도표에 적합
- 코딩테스트에서 문자열 카운팅에 자주 사용

주요 함수

함수/문법	의미
m[key]	value 접근/생성
find(k)	검색
count(k)	존재 여부
erase(k)	삭제
reserve(n)	공간 예약

기본 코드 예시

```
#include <iostream>
#include <unordered_map>
using namespace std;

int main() {
    unordered_map<string, int> cnt;
    cnt["apple"]++;
    cnt["apple"]++;

    cout << cnt["apple"];
}
```

unordered_map 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
삽입/검색/삭제	평균 $O(1)$
최악	$O(n)$
순회	$O(n)$

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	이미 구현된 기능 사용	자료구조 설계와 예외 처리 필요	배열/포인터 직접 관리
코드 길이	짧고 읽기 쉬움	길고 버그 가능성 증가	간단하지만 확장성이 낮음
성능	일반 상황에서 충분히 최적화	구현 실력에 따라 달라짐	고정 크기에서는 단순함
안정성	범위/메모리 관리 부담 감소	메모리/인덱스 실수 가능	범위 초과 위험 큼
추천 상황	대부분의 C++ 코드	자료구조 원리 학습	아주 단순한 고정 크기 데이터

응용: 학생 이름으로 점수 조회

이름을 key로 두고 점수를 value로 저장하면 빠르게 조회할 수 있습니다.

```
#include <iostream>
#include <unordered_map>
using namespace std;

int main() {
    unordered_map<string, int> score = {"Kim", 90}, {"Lee", 85};

    string name = "Kim";
    if (score.find(name) != score.end()) {
        cout << score[name];
    }
}
```

처음 배울 때 자주 하는 실수

- 순회 순서에 의존하면 안 됨
- `m[key]`는 없던 key를 새로 만듦
- 정렬된 출력이 필요하면 `map` 사용

비슷한 항목과 차이

- map은 key 정렬 보장
- unordered_set은 value만 저장
- vector<pair>는 작은 데이터에서 단순할 수 있음

한 줄 정리: unordered_map은(는) '정렬이 필요 없는 빈도수 세기' 상황에서 먼저 떠올리면 좋은 STL입니다.